Explicación de AutoRent para cuatro integrantes

Defensa del proyecto de Gestión de Servidores Web. Fecha de entrega 11 de noviembre de 2026. Duración sugerida de 10 a 12 minutos, más preguntas. Las partes siguientes son una distribución propuesta para exponer; los resultados que se mencionen deben coincidir con el acta de verificación.

Dereck Enrique Ventura Benítez

Tema a cargo: negocio, problema, objetivos y alcance. Tiempo sugerido: 2 a 3 minutos. Mostrar la portada y la flota de AutoRent.

Texto para explicar

Nuestro proyecto se llama AutoRent y está dirigido al alquiler de vehículos de Rent a Car Molina. El negocio presenta unidades para clientes de La Unión y San Miguel. Elegimos este tema porque permite aplicar los contenidos del curso a una operación concreta: consultar un vehículo y reservarlo durante unas fechas.

La aplicación centraliza la información de la flota. El cliente puede revisar fotografías y características, consultar la disponibilidad y obtener una cotización. Después de ingresar a su cuenta, aporta los requisitos y confirma una reserva. El servidor vuelve a comprobar la información antes de guardarla, porque una unidad no puede alquilarse a dos personas durante el mismo período.

La parte principal de Gestión de Servidores Web consiste en cómo se ejecuta esa aplicación. Si una sola instancia deja de funcionar, el usuario pierde acceso a sus operaciones. Por eso preparamos tres instancias de la API, un balanceador que distribuye las solicitudes y una base de datos compartida. También usamos HTTPS y contenedores para reproducir el entorno.

Es importante distinguir la web pública del laboratorio. La web pública ya existía y utiliza su propio alojamiento. Para demostrar las tecnologías de la asignatura utilizamos una copia local conectada a Express y MySQL. Así podemos explicar y verificar cada componente del recorrido.

Nuestro objetivo es mostrar una aplicación útil y una infraestructura que podamos iniciar, comprobar y documentar. La próxima entrega se prepara con el contenido recibido hasta la clase 32. El alcance es únicamente alquiler y el equipo está formado por cuatro integrantes, según lo autorizado por el docente.

Transición

Ahora Andersson explicará qué hace la aplicación cuando un cliente consulta o confirma una reserva y por qué los tres servidores necesitan compartir la misma información.

Andersson Wilfredo Vargas Reyes

Tema a cargo: aplicación, rutas dinámicas y consistencia. Tiempo sugerido: 2 a 3 minutos. Mostrar una ficha y las rutas principales del diseño.

Texto para explicar

AutoRent se divide en una interfaz web y una API. La interfaz es la parte que utiliza el cliente para revisar la flota y completar formularios. La API recibe las solicitudes, aplica las reglas del negocio y consulta MySQL. Está construida con Express y TypeScript.

Una ruta dinámica devuelve información obtenida o calculada por la aplicación. Por ejemplo, la ruta de vehículos consulta el catálogo, la de disponibilidad informa los períodos ocupados y la de cotización calcula los días y el total. Confirmar una reserva es una operación de escritura que requiere una sesión válida y comprobaciones adicionales.

Los tres nodos ejecutan el mismo código. Una petición puede llegar al primero y la siguiente al tercero. Por eso no guardamos las reservas únicamente en la memoria de una instancia: todas consultan la misma base MySQL. También comparten la configuración de autenticación para reconocer las sesiones de forma consistente.

Cuando dos solicitudes intentan reservar el mismo vehículo durante unas fechas que se solapan, el servidor utiliza una transacción y bloqueos en la base. Se vuelve a revisar la disponibilidad al confirmar. La cotización que el cliente vio antes no basta para garantizar la reserva si otra persona confirmó primero.

También se valida el precio en el servidor, se revisan los permisos y se controla el documento del conductor. En el laboratorio utilizamos datos e imágenes ficticios. El kilometraje de ejemplo y los puntos de entrega demostrativos sirven para probar el flujo y no certifican los datos reales del negocio.

Esta estructura permite separar la lógica del alquiler de la forma en que se distribuyen las peticiones. A continuación Luis mostrará cómo Apache, Nginx y Docker conectan estas partes.

Idea que debe quedar clara

Mismo código y misma base permiten que las réplicas atiendan solicitudes sin mantener versiones independientes de las reservas.

Luis Román Lino Merino

Tema a cargo: arquitectura, Docker y HTTPS. Tiempo sugerido: 2 a 3 minutos. Mostrar el diagrama y el estado de los servicios.

Texto para explicar

En el diagrama, el usuario entra por Apache mediante HTTPS. Apache entrega los archivos de la interfaz y envía las solicitudes de la API a Nginx, también mediante HTTPS. Nginx distribuye esas solicitudes entre nodo1, nodo2 y nodo3. Las tres instancias consultan la base MySQL compartida.

Docker permite empaquetar la aplicación con sus dependencias. El Dockerfile describe cómo construir la imagen de la API. Docker Compose describe qué servicios necesitamos, sus redes, sus variables y sus dependencias de inicio. En lugar de abrir cada servidor manualmente, la configuración permite reproducir el conjunto.

La base se inicia antes de las API. Un servicio de inicialización aplica las migraciones y prepara una base exclusiva del laboratorio. Después se levantan los tres nodos. Todos parten de la misma imagen, pero tienen nombres diferentes para que podamos identificar cuál respondió.

La configuración publica la web en el puerto local 8443. Los puertos de diagnóstico de los nodos solo se abren en esta computadora. MySQL no publica su puerto al exterior. La API corre como usuario sin privilegios de administrador y su cuenta de base de datos se limita a operaciones de datos.

Los certificados usados son locales y autofirmados, como admite el curso. Sirven para cifrar las conexiones de la demostración. Para un sitio comercial se necesitarían certificados públicos y una revisión de la infraestructura completa.

Elegimos tres nodos porque la clase 32 los pide para la arquitectura. Esto no significa que tengamos tres computadoras físicas: los contenedores siguen dependiendo del mismo equipo. Tampoco significa que MySQL o el balanceador sean redundantes. Esa limitación forma parte de lo que debemos explicar con precisión.

Transición

Edwin mostrará cómo comprobamos la salud, qué sucede al detener un nodo y qué resultados podemos sostener con la evidencia disponible.

Edwin David Díaz Acosta

Tema a cargo: pruebas, demostración y límites. Tiempo sugerido: 2 a 3 minutos. Mostrar el acta y ejecutar la secuencia cuando el laboratorio esté disponible.

Texto para explicar

La verificación tiene dos partes. Primero revisamos el software mediante TypeScript, compilación y pruebas automatizadas. Después comprobamos la ejecución: conexiones HTTPS, catálogo, salud de las réplicas y comportamiento cuando detenemos una instancia. El acta registra qué comprobaciones se ejecutaron y cuáles quedan pendientes.

La ruta de salud del clúster devuelve el nombre del nodo y verifica MySQL antes de declarar que está saludable. El verificador consulta los tres puertos de diagnóstico. Si detenemos nodo2, debe informar que ese nodo está caído y que los otros dos siguen respondiendo.

Para probar la continuidad no basta con observar que dos contenedores permanecen encendidos. También debemos enviar solicitudes al balanceador y comprobar que las atienden las instancias restantes. Después iniciamos nodo2 y verificamos su recuperación. Si se produce algún error durante la detección, debemos registrarlo en lugar de ocultarlo.

Nginx Open Source usa comprobaciones pasivas en esta configuración. El verificador y los healthchecks de Docker observan la salud por URL. Son mecanismos relacionados, pero no son lo mismo. Un contenedor marcado como no saludable no se reinicia automáticamente solo por ese estado.

La meta de 99.9 por ciento de disponibilidad es un objetivo de diseño. Una demostración de unos minutos no prueba ese porcentaje durante un año. Tampoco hemos eliminado las fallas de la computadora, de Apache, de Nginx o de la base compartida.

La entrega incluye el código, el diagrama, los manuales, el plan de pruebas y los resultados disponibles. Con ello podemos reproducir la instalación y explicar tanto el avance logrado como las tareas que corresponden a la continuación del curso.

Secuencia de demostración

1. Mostrar la flota por HTTPS y consultar /health.
2. Ejecutar scripts/verificar-cluster.ps1.
3. Enviar varias solicitudes por el balanceador y observar los nombres de nodo.
4. Ejecutar docker compose stop nodo2 y repetir las comprobaciones.
5. Ejecutar docker compose start nodo2 y verificar la recuperación.

Preguntas para practicar entre los cuatro

Por qué se necesita una base compartida. Porque una solicitud puede llegar a cualquier réplica y todas deben consultar el mismo inventario y las mismas reservas.

Qué diferencia hay entre Apache y Nginx en este proyecto. Apache sirve la interfaz y actúa como entrada HTTPS. Nginx distribuye las solicitudes de la API entre las tres instancias y también termina una conexión HTTPS.

Qué diferencia hay entre una imagen y un contenedor. La imagen es el paquete construido; el contenedor es una instancia de ese paquete en ejecución. Los nodos usan la misma imagen.

Qué comprueba la ruta de salud. La ruta /health verifica la dependencia MySQL e informa el nodo. La ruta /api/v1/health informa que el proceso responde.

Qué ocurre si falla la base. Las tres API pierden una dependencia común y deben informar indisponibilidad. Replicar las API no hace redundante a MySQL.

Tres nodos significan quórum. En esta API no hay elección de líder. La mayoría de tres procesos no implementa un protocolo de consenso por sí sola.

Qué evita una reserva duplicada. La transacción, la revisión de solapamientos y el control de idempotencia en la base compartida; el número de servidores por sí solo no lo evita.

Está probado el 99.9 por ciento. Es una meta de diseño. Su medición requiere observación y un período definido; el acta solo acredita las pruebas descritas.

Por qué no basta con abrir la web pública. Porque el curso exige demostrar tecnologías concretas y la infraestructura debe poder inspeccionarse y reproducirse.

Qué se entrega mañana. La documentación de esta versión, el repositorio con el código y la configuración, el diagrama de tres nodos y la evidencia disponible de integración. La clase 32 es el último material recibido.

Recomendación para el ensayo

Cada integrante debe poder dibujar el recorrido navegador, Apache, Nginx, API y MySQL sin leerlo. Ensayen una vez con el guion y otra con el diagrama. Mantengan abierto el acta para que las afirmaciones sobre pruebas coincidan con los resultados registrados.